

Algoritmos e Estruturas de Dados II

Seleção e estatísticas de ordem

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

O problema da seleção

O caso mais simples: o menor

O segundo menor

Mediana e k -ésimo menor por força bruta

Quickselect

Análise do quickselect

Mediana das medianas

Achar o **menor** elemento — e por que $n - 1$ comparações bastam



Achar o **segundo menor** sem pagar duas varreduras cheias



A **mediana** e o **k -ésimo menor**: soluções por força bruta



Quickselect: uma partição, um lado só, $O(n)$ em média



Mediana das medianas: $O(n)$ garantido no pior caso

O problema da seleção

Problema

Muitas perguntas não pedem *todos* os elementos em ordem — pedem **um** elemento numa posição específica:

- Qual o **salário mediano** desta empresa?
- Qual a **latência do percentil 99** do servidor?
- Quais são os **10 maiores** arquivos do disco?

Solução

Ordenar o vetor inteiro responde a todas elas — e custa $O(n \log n)$.

Mas ordenar produz n respostas quando queremos **uma**: pagamos caro por informação que vamos jogar fora.

Pergunta da aula

Dá para achar o k -ésimo menor em tempo **linear**?

Definição

Seja $A[0..n-1]$ um vetor com n elementos de um conjunto totalmente ordenado. A **k -ésima estatística de ordem** de A é o elemento que ocuparia a posição k se A fosse ordenado de forma não-decrescente, para $1 \leq k \leq n$.

Intuição

Imagine todo mundo enfileirado do menor para o maior. A k -ésima estatística de ordem é simplesmente *quem está na k -ésima posição da fila*. O problema da **seleção** é descobrir esse elemento **sem precisar formar a fila inteira**.

- $k = 1$: o **mínimo**. $k = n$: o **máximo**.
- $k = \lfloor (n+1)/2 \rfloor$: a **mediana inferior**.

31	8	17	4	25	12	20	6	29
----	---	----	---	----	----	----	---	----

4	6	8	12	17	20	25	29	31
---	---	---	----	----	----	----	----	----

 $k = 9$

máximo

6/52

Quando n é ímpar

Há um elemento exatamente no meio: a mediana é a estatística de ordem $k = (n + 1)/2$. Para $n = 9$, $k = 5$.

Quando n é par

Há *duas* candidatas: a **mediana inferior** ($k = n/2$) e a **mediana superior** ($k = n/2 + 1$). A estatística costuma definir a mediana como a média das duas; em algoritmos, adotamos a **inferior**, com $k = \lfloor (n + 1)/2 \rfloor$.

Por que isso importa

Um algoritmo de seleção resolve *qualquer* k . A mediana é apenas o caso $k \approx n/2$ — e, como veremos, é justamente o caso que quebra as soluções ingênuas.

O caso mais simples: o menor



Invariante

Antes de examinar $A[i]$, a variável `menor` guarda o mínimo de $A[0..i-1]$. Ao final do laço, $i = n$ e `menor` guarda o mínimo de $A[0..n-1]$.

```
1  /* Devolve o indice do menor elemento de A[0..n-1]. */
2  int indice_do_menor(const int A[], int n) {
3      int menor = 0;                /* candidato: A[0] */
4      for (int i = 1; i < n; i++)    /* n-1 iteracoes */
5          if (A[i] < A[menor])      /* 1 comparacao */
6              menor = i;
7      return menor;
8  }
```

Custo: exatamente $n - 1$ comparações entre elementos, em *qualquer* entrada.
Tempo $\Theta(n)$, espaço $O(1)$.

Pergunta: dá para fazer com menos de $n - 1$ comparações?

$n - 1$ comparações é o mínimo possível

Argumento do torneio

Pense em cada comparação como uma *partida* entre dois elementos: o menor vence. Para declarar um campeão, todos os outros $n - 1$ elementos precisam ter **perdido pelo menos uma partida** — caso contrário, um elemento nunca comparado poderia ser menor que o campeão anunciado.

Consequência

Cada comparação produz **no máximo um novo perdedor**. Logo são necessárias pelo menos $n - 1$ comparações, e o algoritmo anterior é **ótimo**.

Guarde esta imagem

A metáfora do torneio parece só um truque de prova — mas é ela que vai resolver o **segundo menor** no próximo slide.

O segundo menor

Ideia

Ache o menor ($n - 1$ comparações), risque-o do vetor, e ache o menor de novo entre os $n - 1$ restantes ($n - 2$ comparações).

1ª passada:

31	8	17	4	25	12	20	6	29
----	---	----	---	----	----	----	---	----

 $n - 1 = 8$ comparações

2ª passada:

31	8	17	-	25	12	20	6	29
----	---	----	---	----	----	----	---	----

 $n - 2 = 7$ comparações

Custo

$(n - 1) + (n - 2) = 2n - 3$ comparações. É $\Theta(n)$ — correto, mas **quase duas varreduras completas**. Jogamos fora tudo o que a primeira passada tinha aprendido.

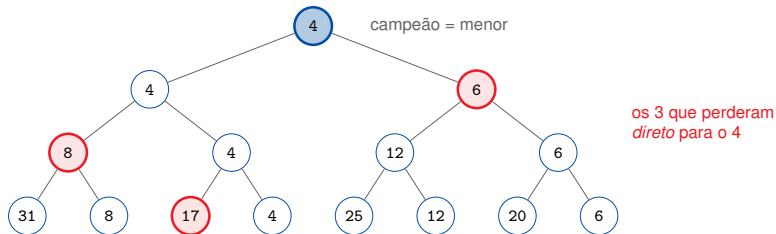
Observação-chave

O segundo menor só pode ter perdido **diretamente para o menor**. Se ele tivesse perdido para qualquer outro elemento y , teríamos $y < \text{segundo menor}$ com $y \neq \text{menor}$ — e y seria o segundo menor, contradição.

Estratégia

Organize as comparações como um **torneio eliminatório**. O campeão (o menor) enfrentou apenas $\lceil \log_2 n \rceil$ adversários. Basta achar o menor **entre esses adversários** — e não entre todos os $n - 1$ perdedores.

$$\underbrace{(n-1)}_{\text{montar o torneio}} + \underbrace{\lceil \log_2 n \rceil - 1}_{\text{menor entre os que perderam para o campeão}} = n + \lceil \log_2 n \rceil - 2$$



Leitura da figura

O campeão 4 venceu 17, depois 8, depois 6: são $\lceil \log_2 8 \rceil = 3$ adversários. O segundo menor é o menor deles — 6 — por apenas 2 comparações extras, contra as 7 da segunda passada ingênua.

O que aprendemos

- O mínimo custa exatamente $n - 1$ comparações, e isso é ótimo.
- O segundo menor sai por $2n - 3$ com força bruta, mas por $n + \lceil \log_2 n \rceil - 2$ se reaproveitarmos as comparações já feitas.
- A lição não é a fórmula: é que **a estrutura das comparações já feitas carrega informação**. Descartá-la é o que torna a força bruta cara.

E agora?

Para $k = 3, k = 4, \dots$ o truque do torneio vai ficando complicado. Precisamos de uma ideia que funcione para **qualquer** k — em especial para $k \approx n/2$, a mediana.

Mediana e k -ésimo menor por força bruta

Ideia: repita k vezes — ache o menor entre os que sobraram e risque-o. É o *selection sort* interrompido na k -ésima rodada.

rodada 1	31	8	17	4	25	12	20	6	29
rodada 2	31	8	17	-	25	12	20	6	29
rodada 3	31	8	17	-	25	12	20	-	29

Custo

$\sum_{j=0}^{k-1} (n - j - 1) = \Theta(kn)$ comparações. Ótimo para k pequeno e constante ($k = 1$, $k = 2$, $k = 3$); $\Theta(n^2)$ **para a mediana**, onde $k = n/2$.

Ideia: x é a k -ésima estatística de ordem se exatamente $k - 1$ elementos são menores que ele. Então, para cada x , conte.

```
1  /* k-esimo menor (1-indexado) por contagem. Assume elementos distintos. */
2  int selecao_forca_bruta(const int A[], int n, int k) {
3      for (int i = 0; i < n; i++) {
4          int menores = 0;
5          for (int j = 0; j < n; j++) /* varredura interna: O(n) */
6              if (A[j] < A[i]) menores++;
7          if (menores == k - 1) /* A[i] tem k-1 abaixo dele */
8              return A[i];
9      }
10     return -1; /* k fora da faixa [1, n] */
11 }
```

Custo: $\Theta(n^2)$ para *qualquer* k — inclusive $k = 1$, onde já sabíamos fazer em $\Theta(n)$. E quebra com elementos repetidos.

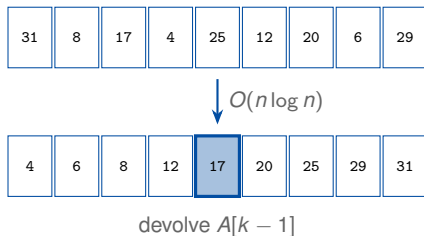
Ideia

Ordene A e devolva $A[k - 1]$.

Custo: $\Theta(n \log n)$ — melhor que $\Theta(n^2)$, e responde a *todos* os k de uma vez.

Quando usar: se você vai fazer muitas consultas sobre o mesmo vetor, ordenar uma vez e indexar é a escolha certa.

O incômodo: para **uma** consulta, ordenar decide a posição relativa de *todos* os pares. Nós queríamos saber a posição de um elemento só.



as outras 8 posições
foram trabalho desperdiçado

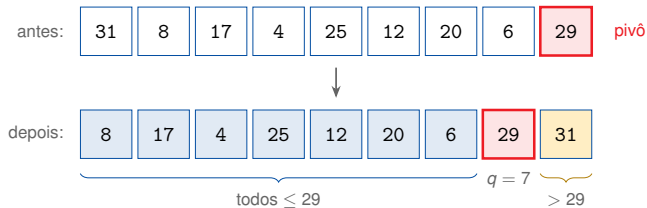
Abordagem	$k = 1$	k constante	mediana ($k \approx n/2$)
Varredura do mínimo	$\Theta(n)$	—	—
Torneio (2º menor)	—	$\Theta(n)$	—
Seleção repetida	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$
Contagem	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Ordenar e indexar	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Meta desta aula	$\Theta(n)$ para todo k		

A pista

Ordenar é caro porque resolve o problema *todo*. Mas o *quicksort* já sabe fazer algo mais barato e muito útil: uma **partição** coloca *um* elemento na sua posição final e separa o resto em “menores” e “maiores”. Talvez uma partição já diga onde está o k -ésimo.

Quickselect

O que uma partição faz: escolhe um **pivô** p e rearranja $A[e..d]$ de modo que p termine numa posição q , com todos os elementos $\leq p$ à esquerda e todos os $> p$ à direita.



Custo

Uma passada: $\Theta(d - e + 1)$ — **linear no tamanho do trecho**. Nenhuma recursão ainda.

Depois de particionar, q não é um índice qualquer

O pivô está na sua **posição final**: há exatamente q elementos $\leq$ a ele à esquerda, logo o pivô é a $(q + 1)$ -ésima estatística de ordem de $A[0..n - 1]$ (índices a partir de 0).

Queremos a k -ésima. Compare k com $q + 1$:

Se...	então...
$k = q + 1$	achamos! O pivô é a resposta. Pare.
$k < q + 1$	a resposta está à esquerda : busque a k -ésima em $A[e..q - 1]$.
$k > q + 1$	a resposta está à direita : busque a $(k - q - 1)$ -ésima em $A[q + 1..d]$.

Quicksort vs. quickselect: o quicksort recursa nos **dois** lados; o quickselect **descarta um deles**. É a única diferença — e é ela que troca $O(n \log n)$ por $O(n)$.

Por que o índice muda ao ir para a direita



Regra

O subvetor da direita não começa mais no início do vetor. Tudo o que ficou à esquerda (inclusive o pivô) **já é menor** que qualquer coisa lá dentro, então esses $q + 1$ elementos precisam ser **descontados** do posto procurado: $k \leftarrow k - (q + 1)$.

Erro clássico: esquecer o desconto e recursar à direita ainda procurando a k -ésima. O programa não trava — devolve silenciosamente o elemento errado.

```
1 static void troca(int *a, int *b) { int t = *a; *a = *b; *b = t; }
2
3 /* Particiona A[e..d] usando A[d] como pivô.
4  Devolve q tal que A[e..q-1] <= A[q] < A[q+1..d]. */
5 int particiona(int A[], int e, int d) {
6     int pivo = A[d];
7     int i = e - 1;                /* fim da regioa dos "<= pivo" */
8     for (int j = e; j < d; j++)
9         if (A[j] <= pivo) {
10             i++;
11             troca(&A[i], &A[j]);
12         }
13     troca(&A[i + 1], &A[d]);      /* pivô vai para sua posicao final */
14     return i + 1;
15 }
```

Invariante do laço: $A[e..i] \leq \text{pivô}$ e $A[i + 1..j - 1] > \text{pivô}$. Ao sair, basta trocar o pivô com $A[i + 1]$.

```
1  /* Devolve a k-esima estatística de ordem (1-indexada) de A[e..d].
2  ATENCAO: reordena A. Pre-condicao: 1 <= k <= d - e + 1. */
3  int quickselect(int A[], int e, int d, int k) {
4      if (e == d) /* um elemento: e ele */
5          return A[e];
6
7      int q = particiona(A, e, d);
8      int posto = q - e + 1; /* posto do pivo dentro de A[e..d] */
9
10     if (k == posto)
11         return A[q]; /* achou */
12     else if (k < posto)
13         return quickselect(A, e, q - 1, k); /* so a esquerda */
14     else
15         return quickselect(A, q + 1, d, k - posto); /* so a direita */
16 }
```

Compare com o quicksort: ele chamaria as *duas* recursões, sem o if.

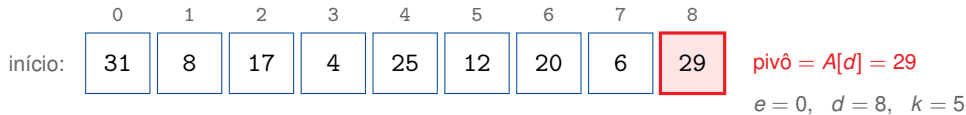
```
1  /* Mesma semantica, sem recursao: a chamada e' de cauda, vira laco. */
2  int quickselect_iter(int A[], int n, int k) {
3      int e = 0, d = n - 1;
4      while (e < d) {
5          int q = particiona(A, e, d);
6          int posto = q - e + 1;
7          if (k == posto) return A[q];
8          if (k < posto) d = q - 1;           /* fica so com a esquerda */
9          else          { k -= posto;        /* desconta o que ficou para tras */
10                     e = q + 1; }          /* fica so com a direita */
11      }
12      return A[e];
13  }
```

Vantagem: espaço auxiliar $O(1)$, sem pilha. A recursão do quickselect é sempre **de cauda** — uma só chamada, e é a última coisa que se faz — então virar laço é mecânico.

Exemplo: a mediana de um vetor de 9 elementos

Instância

$A = [31, 8, 17, 4, 25, 12, 20, 6, 29]$, com $n = 9$. Queremos a mediana: $k = 5$.



Convenção da figura



região ainda em jogo



pivô da rodada

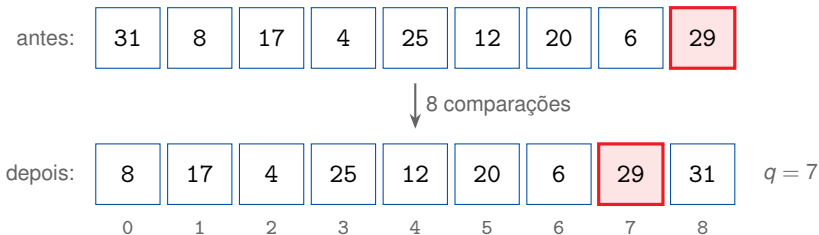


descartada



resposta

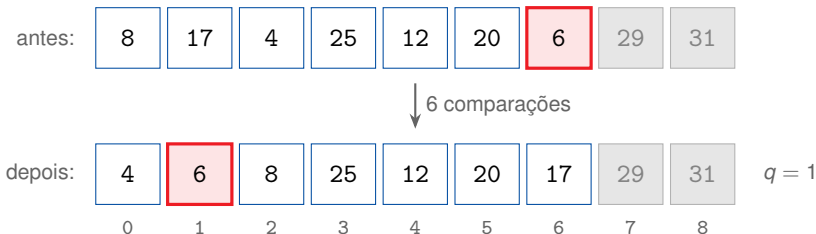
Passo 1: particionar com pivô 29



Decisão

Posto do pivô em $A[0..8]$: $q - e + 1 = 7 - 0 + 1 = 8$. Como $k = 5 < 8$, a mediana está à **esquerda**. Novo trecho: $A[0..6]$, com $k = 5$ inalterado. **Descartamos 2 dos 9 elementos.**

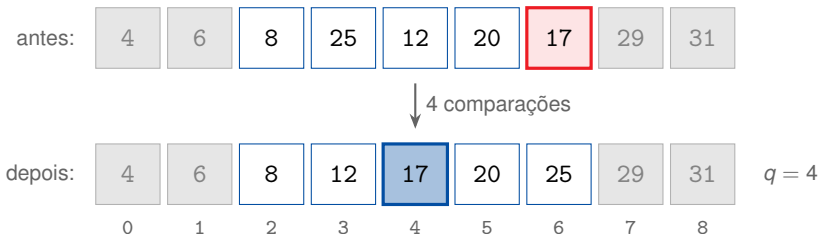
Passo 2: particionar $A[0..6]$ com pivô 6



Decisão

Posto do pivô em $A[0..6]$: $q - e + 1 = 1 - 0 + 1 = \mathbf{2}$. Como $k = 5 > 2$, a mediana está à **direita**. Novo trecho: $A[2..6]$, e o posto vira $k \leftarrow 5 - 2 = \mathbf{3}$.

Passo 3: particionar $A[2..6]$ com pivô 17



Decisão

Posto do pivô em $A[2..6]$: $q - e + 1 = 4 - 2 + 1 = 3$. Como $k = 3 = 3$, **achamos**: a mediana é $A[4] = 17$.

O que o quickselect gastou

- Passo 1: 8 comparações (trecho de 9)
- Passo 2: 6 comparações (trecho de 7)
- Passo 3: 4 comparações (trecho de 5)

O que as forças brutas gastariam

- Contagem: $n^2 = 81$
- Seleção repetida: $8+7+6+5+4 = 30$
- Ordenar: $\approx n \log_2 n \approx 29$

Total: 18 comparações.

Repare no padrão

Os trechos encolheram: $9 \rightarrow 7 \rightarrow 5$. Cada partição custa proporcional ao trecho *atual*, e o trecho só diminui — a conta total é uma **soma decrescente**, e é daí que vai sair o $O(n)$. Mas $9 \rightarrow 7 \rightarrow 5$ está longe de “metade a cada passo”: a análise precisará lidar com partições ruins.

Análise do quickselect

Se todo pivô caísse exatamente no meio, cada partição cortaria o trecho pela metade e o trabalho total seria:



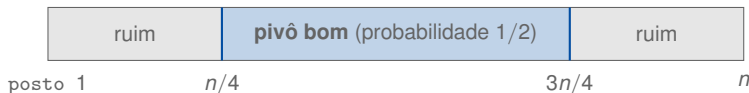
$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n = O(n).$$

O ponto essencial

O quicksort tem $\log n$ níveis de trabalho $\Theta(n)$ cada; o quickselect tem *um* caminho, cujo trabalho **encolhe geometricamente**.

Chamando um pivô de “bom”

Diga que o pivô é **bom** se ele cai na **metade central** do trecho, isto é, se seu posto está entre $n/4$ e $3n/4$. Nesse caso, qualquer que seja o lado escolhido, o trecho restante tem no máximo $3n/4$ elementos.



Com pivô aleatório

Metade dos pivôs possíveis é boa, então esperamos sortear um pivô bom a cada 2 tentativas. Cada tentativa custa $O(n)$, então o custo esperado até *conseguir* encolher para $3n/4$ é $O(n)$.

Recorrência

Absorvendo na constante o custo esperado de chegar a um pivô bom:

$$T(n) \leq T\left(\frac{3n}{4}\right) + cn, \quad T(1) = c.$$

Prova por substituição

Hipótese: $T(m) \leq 4cm$ para todo $m < n$. **Passo:**

$$T(n) \leq T\left(\frac{3n}{4}\right) + cn \leq 4c \cdot \frac{3n}{4} + cn = 4cn. \quad \blacksquare$$

A hipótese se sustenta, logo $T(n) = O(n)$.

Confira pelo teorema mestre: $a = 1$, $b = 4/3$, $n^{\log_b a} = 1$ e $f(n) = \Theta(n)$ — **caso 3**, f domina, logo $T(n) = \Theta(n)$.

Quando dá errado

Se todo pivô for o maior (ou o menor) elemento do trecho, a partição descarta **um único** elemento por rodada:

$$T(n) = T(n-1) + cn = cn + c(n-1) + \dots + c = c \frac{n(n+1)}{2} = \Theta(n^2).$$



nenhuma metade é descartada: encolhe de 1 em 1

Como isso acontece na prática

Com a partição de Lomuto usando $A[d]$ como pivô, basta o vetor chegar **já ordenado** e pedirmos $k = 1$. E vetores quase ordenados são *comuns* em dados reais.

```
1  #include <stdlib.h>
2
3  /* Sorteia o pivô em [e, d] e o joga para a posicao d antes de particionar. */
4  int particiona_aleatorio(int A[], int e, int d) {
5      int r = e + rand() % (d - e + 1);
6      troca(&A[r], &A[d]);
7      return particiona(A, e, d);
8  }
```

O que muda

O pior caso $\Theta(n^2)$ **continua existindo**, mas deixa de depender da *entrada* e passa a depender dos sorteios — nenhum adversário consegue forjar um vetor ruim, porque não conhece os sorteios. O custo esperado passa a ser $O(n)$ para **toda** entrada.

O que aprendemos

- Uma partição já revela a estatística de ordem *exata* do pivô.
- Comparando k com o posto do pivô, sabemos qual lado descartar — e ao ir para a direita, descontamos $q + 1$ de k .
- Caso médio $\Theta(n)$: o trabalho encolhe geometricamente, $T(n) \leq T(3n/4) + cn$.
- Pior caso $\Theta(n^2)$; pivô aleatório o torna improvável, mas não impossível.

A pergunta que sobra

E se eu *precisar* de uma garantia? Sistemas de tempo real e provas de complexidade não aceitam “provavelmente rápido”. Dá para **escolher** um pivô comprovadamente bom, sem sorteio?

Mediana das medianas

Problema

O quickselect é $O(n)$ *em média*, mas ninguém proíbe uma execução $\Theta(n^2)$. Inaceitável quando:

- há prazo rígido (tempo real, controle);
- ele é sub-rotina de uma prova de complexidade — “ $O(n)$ esperado” não fecha o argumento;
- as entradas são adversariais.

Solução

O sorteio dá um pivô bom *com alta probabilidade*. Queremos um pivô bom **com certeza**.

A ideia audaciosa

Um pivô bom é aproximadamente a mediana. Então vamos **calcular uma aproximação da mediana** — usando o próprio algoritmo de seleção, recursivamente.

Algoritmo de Blum, Floyd, Pratt, Rivest e Tarjan (1973) — o “BFPRT”, ou **mediana das medianas**.

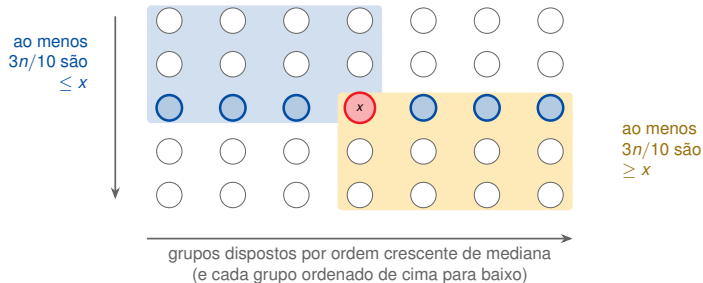
Como escolher o pivô de $A[e..d]$ com n elementos

1. Divida os n elementos em $\lceil n/5 \rceil$ **grupos de 5** (o último pode ser menor).
2. Ordene cada grupo e pegue sua **mediana**. Cada grupo tem tamanho fixo 5, então isso custa $O(1)$ por grupo, $O(n)$ no total.
3. Chame o **próprio algoritmo de seleção**, recursivamente, para achar a mediana x dessas $\lceil n/5 \rceil$ medianas.
4. Use x como pivô e siga o quickselect normalmente.

Por que 5?

Precisamos que a soma das frações das duas recursões seja < 1 . Com grupos de 5: $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$ — funciona. Com grupos de 3: $\frac{1}{3} + \frac{2}{3} = 1$ — não funciona. Grupos de 7 também funcionam, mas 5 dá a melhor constante.

Por que a mediana das medianas é um pivô bom



Leitura da figura

Cada grupo à esquerda de x tem mediana $\leq x$; nesses grupos, os 3 elementos do topo são $\leq$ à sua mediana, logo $\leq x$. São $3 \cdot \lceil n/10 \rceil$ elementos. Simetricamente à direita.

Limite

Ao menos $3n/10 - 6$ elementos são $\leq x$ (o “-6” absorve arredondamentos e o grupo incompleto). Logo o lado que **sobrevive** tem no máximo $n - (\frac{3n}{10} - 6) = \frac{7n}{10} + 6$ elementos.

Recorrência

$$T(n) \leq \underbrace{T\left(\left\lceil \frac{n}{5} \right\rceil\right)}_{\text{achar a mediana das medianas}} + \underbrace{T\left(\frac{7n}{10} + 6\right)}_{\text{continuar a seleção}} + \underbrace{O(n)}_{\text{formar grupos e particionar}}$$

Atenção

São duas recursões e ainda assim o resultado é linear. O teorema mestre não se aplica — os subproblemas têm tamanhos diferentes.

Método da substituição

Hipótese: $T(m) \leq am$ para todo $m < n$, com a a escolher.

Passo (ignorando os termos constantes por simplicidade):

$$T(n) \leq a \frac{n}{5} + a \frac{7n}{10} + cn = \frac{9an}{10} + cn = an - \left(\frac{an}{10} - cn \right).$$

Basta que $\frac{an}{10} - cn \geq 0$, isto é, $a \geq 10c$. Com essa escolha, $T(n) \leq an$ e portanto $T(n) = O(n)$. ■

A “folga” $\frac{1}{10}$ que sobra de $1 - \frac{9}{10}$ é exatamente o que paga o trabalho linear de cada nível.

```
1  int selecao(int A[], int e, int d, int k); /* declaracao antecipada */
2  /* Escolhe um pivo bom para A[e..d] e devolve seu indice. */
3  int pivo_mediana_das_medianas(int A[], int e, int d) {
4      int n = d - e + 1;
5      if (n <= 5) { insertion_sort(A, e, d); return e + n / 2; }
6
7      int g = 0;                                /* quantos grupos ja' processei */
8      for (int i = e; i <= d; i += 5) {
9          int fim = (i + 4 < d) ? i + 4 : d;
10         insertion_sort(A, i, fim);              /* grupo de <= 5: O(1) */
11         int mediana = i + (fim - i) / 2;
12         troca(&A[e + g], &A[mediana]);          /* junta as medianas em A[e..] */
13         g++;
14     }
15     /* mediana das g medianas, que agora ocupam A[e .. e+g-1] */
16     selecao(A, e, e + g - 1, (g + 1) / 2);
17     return e + (g - 1) / 2;
18 }
```


O algoritmo de seleção linear completo



```
1  /* Particiona A[e..d] em torno do elemento que esta' em A[p]. */
2  int particiona_em(int A[], int e, int d, int p) {
3      troca(&A[p], &A[d]);
4      return particiona(A, e, d);
5  }
6
7  /* k-esima estatistica de ordem de A[e..d]: O(n) no PIOR caso. */
8  int selecao(int A[], int e, int d, int k) {
9      if (e == d) return A[e];
10
11     int p = pivo_mediana_das_medianas(A, e, d);  /* T(n/5) */
12     int q = particiona_em(A, e, d, p);          /* O(n) */
13     int posto = q - e + 1;
14
15     if (k == posto) return A[q];
16     else if (k < posto) return selecao(A, e, q - 1, k);  /* <= 7n/10 */
17     else return selecao(A, q + 1, d, k - posto);  /* <= 7n/10 */
18 }
```

Algoritmo	Médio	Pior	Comentário
Ordenar e indexar	$n \log n$	$n \log n$	vale a pena se houver muitas consultas
Quickselect (pivô fixo)	n	n^2	quebra em vetor ordenado
Quickselect aleatório	n	n^2	a escolha padrão na prática
Mediana das medianas	n	n	garantia; constante grande
<i>Introsselect</i>	n	n	híbrido: o que a <code>libstdc++</code> usa

A constante importa

A mediana das medianas é linear, mas a constante escondida no $O(n)$ é grande. Para n típico, o quickselect aleatório costuma ser **várias vezes mais rápido**.

Introsselect: rode o quickselect aleatório e **conte as rodadas**; passando de $c \log n$ rodadas, troque para a mediana das medianas.

Menor: uma varredura, $n - 1$ comparações — e isso é ótimo



2º menor: reaproveitar o torneio, $n + \lceil \log n \rceil - 2$



k -ésimo, força bruta: $\Theta(kn)$, $\Theta(n^2)$ ou $\Theta(n \log n)$



Quickselect: uma partição, um lado só — $\Theta(n)$ médio, $\Theta(n^2)$ pior



Mediana das medianas: pivô garantido — $\Theta(n)$ no pior caso

Não resolva mais do que foi perguntado

Ordenar responde a n perguntas quando fizemos 1. Toda a economia do quickselect vem de **jogar fora metade do problema** a cada passo, em vez de resolvê-lo.

Redução do problema (*decrease and conquer*)

Repare no contraste com dividir e conquistar:

- **Quicksort** divide e resolve os **dois** subproblemas $\Rightarrow \log n$ níveis de trabalho $\Theta(n) \Rightarrow \Theta(n \log n)$.
- **Quickselect** divide e resolve **um** subproblema $\Rightarrow$ soma geométrica $\Rightarrow \Theta(n)$.

O mesmo contraste separa a busca binária ($\Theta(\log n)$) de um percurso completo da árvore.

1. Adapte o `quickselect` para devolver o k -ésimo **maior** sem inverter o vetor nem trocar o comparador.
2. O `selecao_forca_bruta` por contagem quebra com elementos repetidos. Conserte-o e explique por que o `quickselect` *não* sofre do mesmo problema.
3. Mostre uma entrada de tamanho n que force o `quickselect` ($\text{pivô} = A[d]$) a fazer $\Theta(n^2)$ comparações para $k = n$.
4. Escreva `k_menores(A, n, k)` que devolve os k menores elementos (em qualquer ordem) em tempo $O(n)$ para k fixo. *Dica: uma chamada ao `quickselect` resolve.*
5. Prove que grupos de 3 *não* produzem um algoritmo linear: escreva a recorrência e mostre onde a prova por substituição falha.
6. Implemente o *introselect* e meça, para $n = 10^6$, quantas vezes o gatilho da mediana das medianas é acionado.

-  Jon Bentley.
Programming Pearls.
Addison-Wesley, 2 edition, 2000.
-  Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan.
Time bounds for selection.
Journal of Computer and System Sciences, 7(4):448–461, 1973.
Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.

-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.
-  Michael R. Garey and David S. Johnson.
Computers and Intractability: A Guide to the Theory of NP-Completeness.
W. H. Freeman, 1979.
-  David Gries.
The Science of Programming.
Springer-Verlag, 1981.

-  Michael Held and Richard M. Karp.
A dynamic programming approach to sequencing problems.
Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.
-  C. A. R. Hoare.
Algorithm 64: Quicksort.
Communications of the ACM, 4(7):321, 1961.
-  C. A. R. Hoare.
Algorithm 65: FIND.
Communications of the ACM, 4(7):321–322, 1961.
O quickselect, publicado por Hoare no mesmo ano do quicksort.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Donald E. Knuth.

The Art of Computer Programming, Volume 3: Sorting and Searching.

Addison-Wesley, 2 edition, 1998.

Seção 5.3.3: seleção mínima em número de comparações.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.
Addison-Wesley, 1989.



David R. Musser.

Introspective sorting and selection algorithms.
Software: Practice and Experience, 27(8):983–993, 1997.
O introselect: híbrido entre quickselect e mediana das medianas.



Robert Sedgewick and Kevin Wayne.

Algorithms.
Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.